

# 深圳大学考试答题纸

(以论文、报告等形式考核专用)

二〇二五 ~ 二〇二六 学年度第 2 学期

《移动互联网应用》课程项目设计，根据如下题目完成项目设计和报告：

题目：我的极简待办清单 APP

## 1. 极简待办清单 APP 界面

➤ 基于已完成的“APP 调研报告”相关的经验总结，尽量模拟如下“极简待办清单”APP（华为应用市场：极简待办清单）的功能，即参考如下的界面展示形式及功能模块：

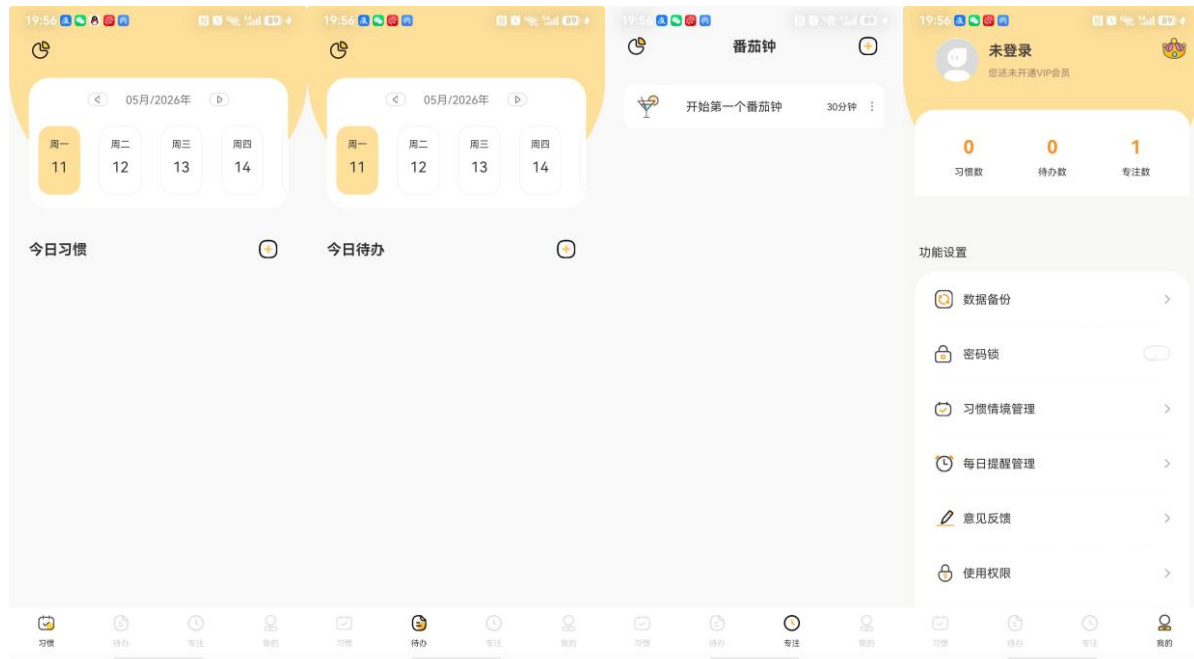


图 1 极简待办清单 APP；功能模块及其高清图可以去 APP 查看

## 2. 具体要求

模拟图 1 所示极简待办清单 APP，介绍该 APP 与服务相关的一些知识点并能提供相应功能：

- 1) 建议包含的一些功能：活动之间的转换与数据传递；能适应不同的展示界面；有登录功能，强制下线功能；数据有多样化的持久化功能；能跨程序提供与共享数据；有展示一些多媒体的功能；
- 2) 较好的实现了书本上介绍的一些较成熟的功能，并能较好的把这些功能融合在一个完整且无大 bug 的 APP 里；
- 3) 能在此基础上构建自己的报告亮点，如实现了书本不一样的功能模块，或者为某个知识点找到适当的新的应用场景，或者能解决同学们普遍存在的一些问题等；
- 4) 模拟的 APP 不局限于所参照 APP 的功能，即只需尽量模拟这些功能，**不要求将每个界面或功能都实现，如果某个界面或功能不能体现已学知识点，可以不用考虑**，当然如果能想办法实现出来，可以作为报告亮点；**即不必与这些界面或功能完全一样，可在此基础上进行变通，达到类似的效果就可以**；如果设计了一些该 APP 没有的功能，请说明清楚这些功能的实现方式、潜在的用途等；
- 5) **总体目标是考察同学们是否已掌握课程内容，是否能灵活利用所学的知识点**，做到每个功能各种实现方式的丰富化（如数据的持久化的三种实现方式都能在 APP 中有所体现），并且能了解并总结不同实现方式的优劣，且能通过 APP 的实现体现出来；

## 3. 部分参考

1) 功能实现参考：图第 4 列尽量参考第 6 章数据持久化技术的各个知识点；第 1 列尽量参考布局及活动之间的跳转，碎片的实现，多媒体展示功能；顶部与底部菜单可以利用 TabLayout、ViewPager 或 RecyclerView 等实现；

2) 设计 Android 基于位置的服务, 比如能根据用户所在位置获取用户的位置动态变化; 利用碎片实现简单的聊天框功能; 添加一个小功能, 整合网络技术的应用, 比如“浏览”、“简介”或“关注”的功能的线下保存; 利用数据后台下载的功能, 比如电子书的下载等;

3) 可以借鉴的部分章节内容, 第 12 章(第二版)可以让你的 APP 界面变得更美观; 第 14 章展示了一个大型的工程, 可以学习下多个功能怎样在一个工程里体现;

4) 添加至少一个人工智能功能模块, 比如推荐算法(根据搜索进行个性化推荐), 识别或分类算法(识别自己的手势或表情);

#### 4. 其它要求

1) 构建的 APP 要格式工整, 美观; 个人补充的新的功能或界面可以与“APP 调研报告”的总结对应上;

2) 实验报告中需要有功能的描述、实验结果的截屏图像及详细说明; 结果展示要具体, 图文交叉解释; 代码与文本重点要突出, 代码的展示要适当少一些; 建议报告采用白色背景的形式;

3) 也欢迎采用课程后续章节的知识点完成本次大作业, 如果实现的功能言之合理, 会考虑酌情加分;

4) 每位同学在最后一次课都需要上台报告, 并且最好能现场演示 APP 的功能等, 没上台报告的同学分数会受一定的影响;

5) 报告由个人独立完成。

#### 5. 评分标准

1) APP 协议完成度高, 与参考 APP 有一定的相似度, 功能完善、丰富。能实现活动的编写、自定义用户界面的开发、碎片开发、广播机制、数据持久化与共享技术、网络技术、后台服务等的应用等(60 分)

2) 模拟 APP 结构合理, 代码规范, 界面美观易用。项目报告撰写规范、美观整齐, 内容详实且能准备描述项目内容和设计思想、原理、框架等, 项目报告要求 5 号字、除前两页试题要求外 A4 版面不少于 10 页的长度, 但也不应超过 45 页。(15 分)

3) 提供程序源代码和可执行程序(或安装程序); 报告文档采用单独的 word 文档, 项目所有代码(不是整个工程文件, 应该总共不超过 5M)在第 17 周结束前打包作为附件进行上传 blackboard 系统; 报告纸质版采用双面打印, 并交到任课老师指定处。(10 分)

4) 项目报告能够详细, 准确的描述项目内容与亮点, 并在最后一堂课有较好的展示效果。(15 分)

## 三、实验过程和代码与结果

### 1. APP 的构建过程及结果

#### 1.1 需求分析

确定功能模块, 将 APP 划分为五个模块, 分别对应五个页面:

##### 板块一: 习惯

功能设计:

①顶部日历栏: 支持折叠/展开双模式(周视图 / 月视图), 左右箭头切换月份或周。默认选中当天。

②日期标记: 日历格子右上角显示当天未完成打卡数量的角标。

③列表展示: 以卡片形式按计划表配置过滤展示当日应出现的习惯。习惯卡片包含彩色圆点、习惯

名称、打卡状态（已完成/未完成）、频率摘要。

④快速打卡：点击习惯卡片上的勾选框，切换打卡状态，日历角标同步刷新。

⑤滑动删除：支持左滑删除习惯（弹出确认对话框）。

⑥添加/编辑：点击新增按钮或习惯卡片跳转，支持创建新习惯与编辑已有习惯。

## 板块二：待办

功能设计：

①顶部日历栏：与习惯模块共用同一日历组件，支持折叠展开与角标显示。

②待办列表：按日期过滤显示，按未完成优先、截止日期、优先级降序排列。

③列表项信息：左侧优先级色带（红=高/橙=中/绿=低），优先级标签，截止日期智能显示（今天/明天/昨天/MM/dd），备注预览。

④勾选完成：点击复选框切换完成状态，已完成项目灰色删除线显示。

⑤滑动删除：左滑弹出确认对话框删除。

⑥新增/编辑待办：点击新增按钮或点击卡片跳转编辑，支持设置标题、截止日期/时间、备注、优先级（低/中/高）。

## 板块三：番茄钟

功能设计：

①状态展示：顶部状态栏显示当前倒计时、阶段标签（工作/短休息/长休息）。运行中显示暂停/停止按钮，空闲时显示默认时长。

②全屏计时：点击状态栏跳转全屏，圆形进度条 + 大字号等宽倒计时数字 + 阶段标签。

③控制项：单一按钮循环开始→暂停→继续→暂停，停止按钮退出计时。

④任务列表：支持添加番茄钟任务，预设时长芯片（25/30/45/60 分钟）选中高亮，或自定义分钟数。

⑤设置页：配置工作时长、短休息时长、长休息时长、长休息间隔周期数。

## 板块四：AI 智能助手

功能设计：

①对话界面：气泡式聊天界面（发送/接收双样式），底部输入框 + 发送按钮，支持清空对话、长按删除单条消息。

②自然语言操作：用户可使用自然语言创建/修改/删除/完成待办、习惯和番茄钟任务，AI 自动执行并反馈结果。

③API 配置：首次使用提示配置 DeepSeek API Key，支持随时在"我的"→"AI 设置"中修改。

## 板块五：我的界面交互说明

功能设计：

①个人信息区：头像（可更换 emoji 头像）、登录状态（点击登录/登出）。

②数据统计面板：三格展示习惯数、待办数、番茄专注完成次数。

③位置卡片：显示当前地址、经纬度、精度。App 启动时自动在后台获取位置，切换到"我的"时即显示最新结果，无需手动刷新。

④设置列表：AI 设置（API Key 配置）、数据备份管理、应用权限入口。

## 1.2 架构设计

为确保系统功能的合理性与扩展性，技术架构采用经典的分层设计：

| Android 应用架构 - 分层设计                                                                        |                            |
|--------------------------------------------------------------------------------------------|----------------------------|
| 表示层 (UI Layer)                                                                             |                            |
| Activity (Splash, Login, Lock, Main, EditTodo, EditHabit, PomodoroTimer, PomodoroSettings) |                            |
| Fragment (Habit, Todo, Pomodoro, Discover/Chat, Mine)                                      |                            |
| RecyclerView, ViewPager2, BottomNavigationView, Custom Views                               |                            |
| 业务逻辑层 (Logic Layer)                                                                        |                            |
| BroadcastReceiver (强制下线, 每日提醒, 通知操作)                                                       |                            |
| Foreground Service (番茄钟倒计时)                                                                | ChatClient (Retrofit 单例)   |
| RecommendationEngine (本地规则推荐)                                                              | AI Command Parser          |
| 数据源与支撑层 (Data Layer)                                                                       |                            |
| Room/SQLite (6 张实体表)                                                                       | SharedPreferences (配置与会话)  |
| File Storage (备份/日志)                                                                       | ContentProvider (跨进程共享)    |
| Location API (FusedLocation)                                                               | Retrofit + OkHttp (AI API) |
| Android Keystore (AES/GCM)                                                                 | AlarmManager (定时提醒)        |

### 1.3 数据库设计

### ①待办数据表

| 字段名          | 类型      | 约束                        | 说明               |
|--------------|---------|---------------------------|------------------|
| id           | INTEGER | PRIMARY KEY AUTOINCREMENT | 主键 ID            |
| title        | TEXT    | NOT NULL                  | 待办名称             |
| note         | TEXT    |                           | 备注详细内容           |
| due_date     | LONG    |                           | 截止日期时间戳          |
| is_completed | INTEGER | DEFAULT 0 (0/1)           | 是否已完成            |
| priority     | INTEGER | DEFAULT 1                 | 优先级（1-低，2-中，3-高） |
| user_id      | TEXT    | DEFAULT ''                | 所属用户（空串为旧数据兼容）   |

### ②习惯数据表

| 字段名             | 类型      | 约束                        | 说明              |
|-----------------|---------|---------------------------|-----------------|
| id              | INTEGER | PRIMARY KEY AUTOINCREMENT | 主键 ID           |
| name            | TEXT    | NOT NULL                  | 习惯名称            |
| description     | TEXT    |                           | 描述              |
| icon_res        | TEXT    |                           | 图标/emoji 标识符    |
| frequency       | TEXT    |                           | 频率标签（“每日”/“每周”） |
| color           | TEXT    |                           | 颜色 hex 值        |
| create_time     | LONG    |                           | 创建时间            |
| schedule_config | TEXT    |                           | JSON 调度配置（见下文）  |

| 字段名     | 类型   | 约束         | 说明   |
|---------|------|------------|------|
| user_id | TEXT | DEFAULT '' | 所属用户 |

### ③习惯打卡表

| 字段名        | 类型      | 约束                        | 说明              |
|------------|---------|---------------------------|-----------------|
| id         | INTEGER | PRIMARY KEY AUTOINCREMENT | 主键 ID           |
| habit_id   | LONG    | NOT NULL                  | 关联习惯 ID         |
| date_stamp | LONG    | NOT NULL                  | 日期标记 (yyyyMMdd) |
| checked    | INTEGER | DEFAULT 0 (0/1)           | 打卡状态            |
| user_id    | TEXT    | DEFAULT ''                | 所属用户            |

### ④番茄钟任务表

| 字段名              | 类型      | 约束                        | 说明       |
|------------------|---------|---------------------------|----------|
| id               | INTEGER | PRIMARY KEY AUTOINCREMENT | 主键 ID    |
| name             | TEXT    |                           | 任务名称     |
| icon             | TEXT    |                           | 图标/emoji |
| duration_minutes | INTEGER |                           | 时长（分钟）   |
| create_time      | LONG    |                           | 创建时间     |
| user_id          | TEXT    | DEFAULT ''                | 所属用户     |

### ⑤番茄钟会话表

| 字段名        | 类型      | 约束                                       | 说明                 |
|------------|---------|------------------------------------------|--------------------|
| id         | INTEGER | PRIMARY KEY AUTOINCREMENT                | 主键 ID              |
| task_id    | LONG    | FK → pomodoro_task(id) ON DELETE CASCADE | 关联任务               |
| start_time | LONG    |                                          | 开始时间戳              |
| end_time   | LONG    | NULLABLE                                 | 结束时间戳 (null = 进行中) |
| completed  | INTEGER | DEFAULT 0 (0/1)                          | 是否完成               |
| user_id    | TEXT    | DEFAULT ''                               | 所属用户               |

### ⑥聊天信息表

| 字段名        | 类型      | 约束                        | 说明                          |
|------------|---------|---------------------------|-----------------------------|
| id         | INTEGER | PRIMARY KEY AUTOINCREMENT | 主键 ID                       |
| content    | TEXT    | NOT NULL                  | 消息内容                        |
| type       | INTEGER |                           | 0=接收到(AI), 1=发送(用户)         |
| timestamp  | LONG    |                           | 时间戳                         |
| session_id | TEXT    | NOT NULL                  | 会话标识 (格式: ai_chat_{userId}) |

1.4 持久化数据存储方式设计

| 数据        | 存储方式                                                   | 说明                                |
|-----------|--------------------------------------------------------|-----------------------------------|
| 用户密码      | SharedPreferences (user_<name>_password)               | SHA-256 哈希值                       |
| 记住密码      | SharedPreferences (rem_pwd_<name>)                     | AES-256/GCM 加密 (Android Keystore) |
| 登录历史      | SharedPreferences (login_history)                      | 去重 LinkedHashSet, 最多 4 个最近用户      |
| 当前用户      | SharedPreferences (logged_in_user)                     | 登录用户名                             |
| 应用锁开关     | SharedPreferences (password_hash, app_unlocked)        | 密码哈希 + 解锁状态                       |
| 番茄钟配置     | SharedPreferences (pomodoro_length 等)                  | 工作/休息时长、周期数、计数                    |
| AI API 配置 | SharedPreferences (deepseek_api_key, deepseek_api_url) | DeepSeek API 密钥和 URL              |
| 头像        | SharedPreferences (avatar_<username>)                  | emoji 字符                          |
| 每日提醒      | SharedPreferences (reminder_*)                         | 提醒开关与时间                           |
| 备份文件      | File Storage (files/backups/ + 公共 Downloads)           | JSON 格式                           |

1.5 确定实现技术路径与开发环境

开发语言：Java+ Kotlin  
构建工具：Gradle 8.x (Kotlin DSL) + Version Catalog ( libs.versions.toml )  
最低 SDK：API 24 (Android 7.0)  
目标/编译 SDK：API 36  
JDK 版本：Java 11 (启用 coreLibraryDesugaring 以支持 java.time 包)

2. 请详细说明 APP 的功能、出现的关键问题及解决方案

2.1 今日习惯

支持创建每日/每周/特定日期三种模式的习惯，可设置打卡提醒时间。顶部日历栏支持周/月视图折叠切换，日历格子右上角显示当天待打卡数量角标。习惯卡片包含彩色圆点、名称、频率摘要，点击复选框快速打卡，左滑删除。

### 问题难点与实现路径：

日历组件：CalendarMonthAdapter 渲染 42 格(6 行 × 7 列)月历网格，通过 updateGrid() 和 updateCounts() 两个独立更新方法避免全量刷新导致的闪烁。

习惯过滤：通过 HabitItem.isActiveOnDate(LocalDate) 方法解析 schedule\_config JSON，判断每日/每周/特定日期模式的习惯是否在选中日期生效。

滑动删除：使用 ItemTouchHelper.SimpleCallback 结合 RecyclerView 实现左滑删除，弹出确认对话框。

```
public void updateCounts(Map<LocalDate, Integer> counts) {
    this.counts = counts;
    notifyDataSetChanged();
}

Integer count = counts != null ? counts.get(date) : null;
if (count != null && count > 0) {
    tvBadge.setText(String.valueOf(Math.min(count, 99))); // max "99"
    tvBadge.setVisibility(View.VISIBLE);
} else {
    tvBadge.setVisibility(View.GONE);
}

holder.itemView.setOnClickListener( View v -> {
    if (listener != null) listener.onDayClick(date);
});
}
```

图 2-1-1、图 2-1-2 日历角标更新相关代码



图 2-1-3 至图 2-1-5 习惯页展示、日历组件与打卡效果

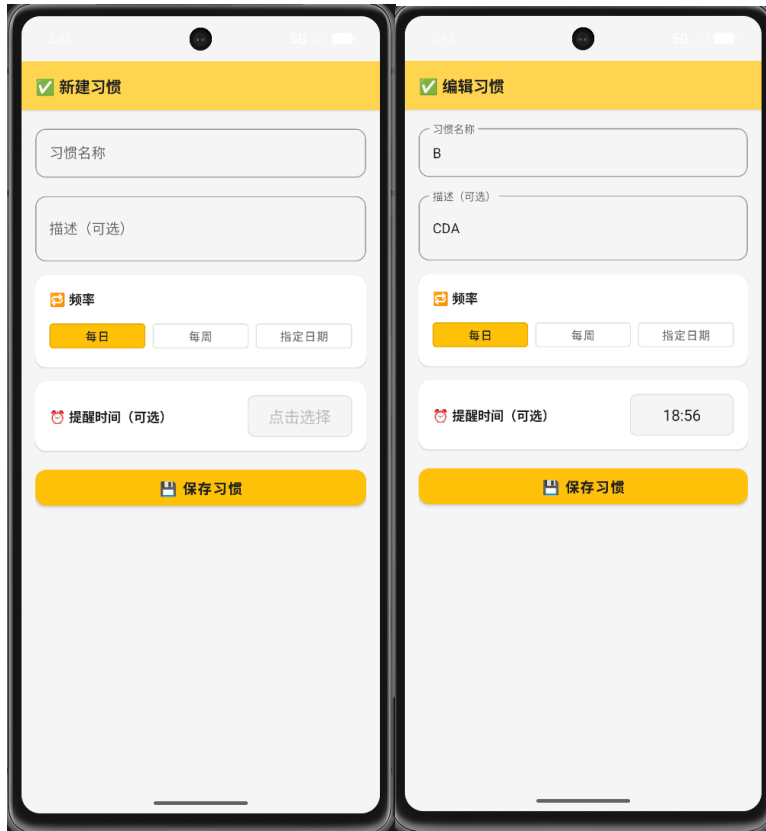


图 2-1-6、图 2-1-7 新建习惯、编辑习惯

## 2.2 今日待办

支持创建带标题、备注、截止日期/时间、优先级（低/中/高）的待办事项。列表按"未完成优先 → 截止日期 → 优先级降序"排列。待办卡片左侧显示优先级色带（红=高/橙=中/绿=低），截止日期智能显示（今天/明天/昨天/MM-dd），已完成项灰色删除线显示。

### 问题难点与实现路径：

优先级 UI：三级 Chip 选择器，点击切换背景和文字颜色。

日期智能显示：DateUtils.formatDueDate(long) 方法计算与今日差值，返回"今天"/"明天"/"昨天"或 MM/dd 格式。

数据传递：通过 Intent Bundle 携带 todo\_id，在 EditTodoActivity 中判断新建/编辑模式。

```
public static String formatDueDate(long timestamp) {
    if (timestamp <= 0) return "";
    java.time.Instant instant = java.time.Instant.ofEpochMilli(timestamp);
    LocalDate date = instant.atZone(java.time.ZoneId.systemDefault()).toLocalDate();
    LocalDate today = LocalDate.now();
    if (date.equals(today)) return "今天";
    if (date.equals(today.plusDays(daysToAdd: 1))) return "明天";
    if (date.equals(today.minusDays(daysToSubtract: 1))) return "昨天";
    return date.format(DateTimeFormatter.ofPattern("MM/dd"));
}
```

图 2-2-1 日期差值计算代码

```
todoAdapter.setOnItemClickListener( TodoItem item -> {
    Intent i = new Intent(getContext(), EditTodoActivity.class);
    if (item.id != null) i.putExtra( name: "todo_id", item.id);
    startActivity(i);
});
```

图 2-2-2 数据传递代码



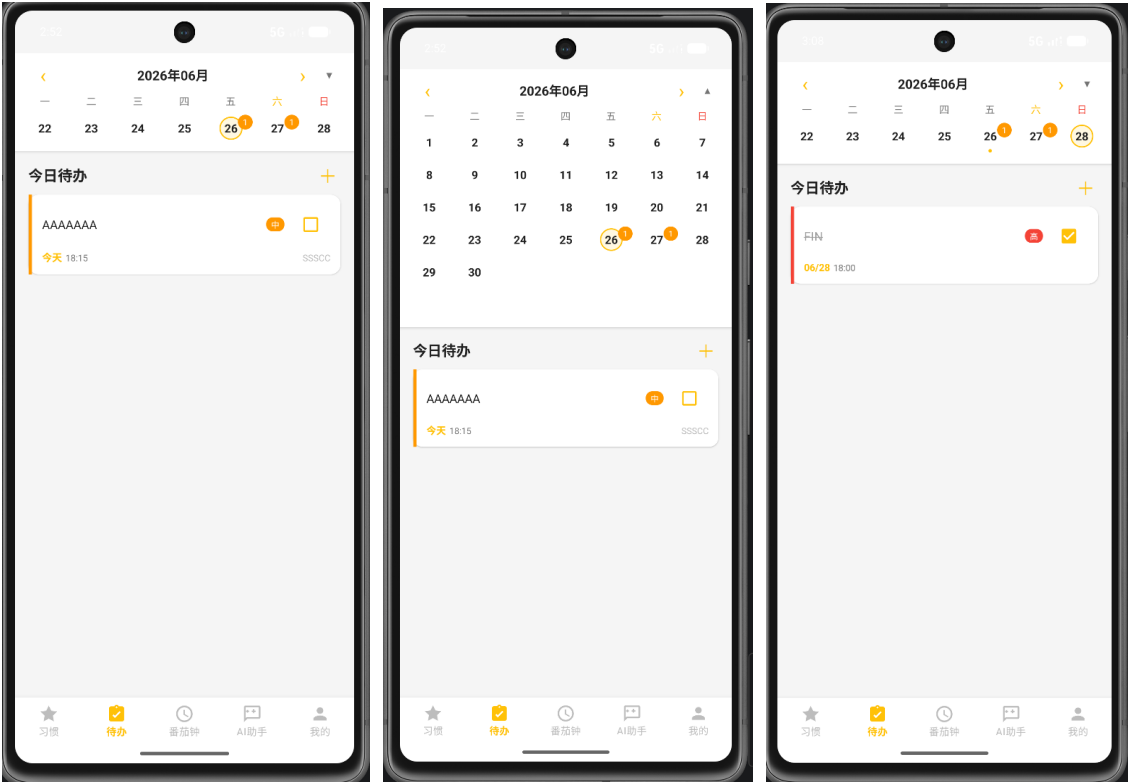


图 2-2-3 至图 2-2-5 待办页展示、日历组件与完成待办效果



图 2-2-6、图 2-2-7 新建、编辑待办

2.3 番茄钟

支持前台服务驱动的倒计时，三段式自动循环（工作 → 短休息 → 长休息 → 工作）。全屏计时界面含圆形进度条和大字号倒计时数字。通知栏实时显示进度条及暂停/停止操作按钮。支持任务列表管理，预设时长芯片（25/30/45/60 分钟）选中高亮。

问题难点与实现路径：

前台服务： PomodoroService 继承 Service ，使用 CountDownTimer 1 秒间隔驱动，START\_STICKY 确保进程死亡后自动恢复。

广播通信：6 种自定义广播（TICK / PAUSED / RESUMED / STOPPED / PHASE\_CHANGED /FINISHED），通过 setPackage() 限制包内广播避免 API 26+ 隐式广播限制。

状态持久化：所有计时状态（running / paused / phase / remaining / total / endTime）写入 SharedPreferences，进程死亡后通过 restoreState() 恢复。

三段式循环： advancePhase() 方法判断当前阶段 → 根据配置的周期数选择下一阶段 → 自动开始新阶段倒计时。

```
private void advancePhase() {
    SharedPreferences p = getSharedPreferences(PREFS, MODE_PRIVATE);

    if (phase == PHASE_WORK) {
        // Work session completed → increment count and go to break
        int count = p.getInt(key: "pomodoro_count", defValue: 0);
        p.edit().putInt("pomodoro_count", count + 1).apply();
        workCount++;

        int cycles = p.getInt(key: "cycles_before_long", defValue: 4);
        if (cycles <= 0) cycles = 4;

        if (workCount % cycles == 0) {
            phase = PHASE_LONG;
            totalMillis = p.getInt(key: "long_break", defValue: 15) * 60L * 1000L;
        } else {
            phase = PHASE_SHORT;
            totalMillis = p.getInt(key: "short_break", defValue: 5) * 60L * 1000L;
        }
    } else {
        // Break finished → go back to work
        phase = PHASE_WORK;
        totalMillis = p.getInt(key: "pomodoro_length", defValue: 25) * 60L * 1000L;
    }
}
```

图 2-3-1 三段式循环代码

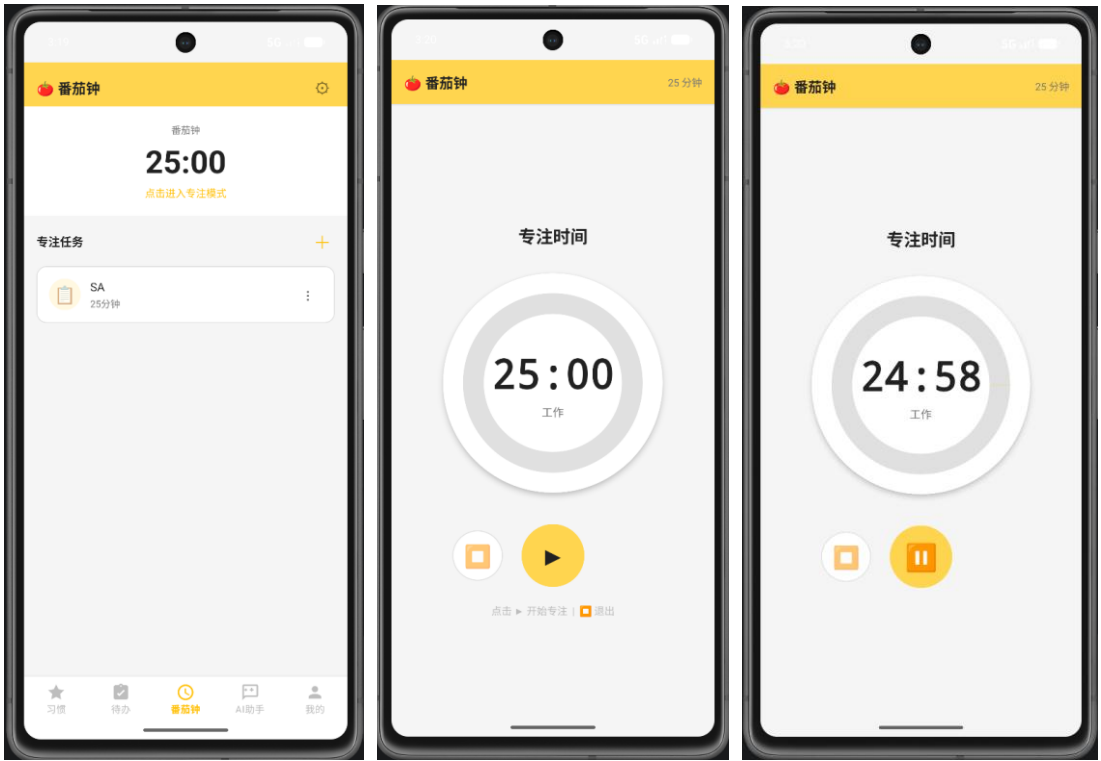


图 2-3-2 至图 2-3-4 番茄钟列表与全屏工作展示页

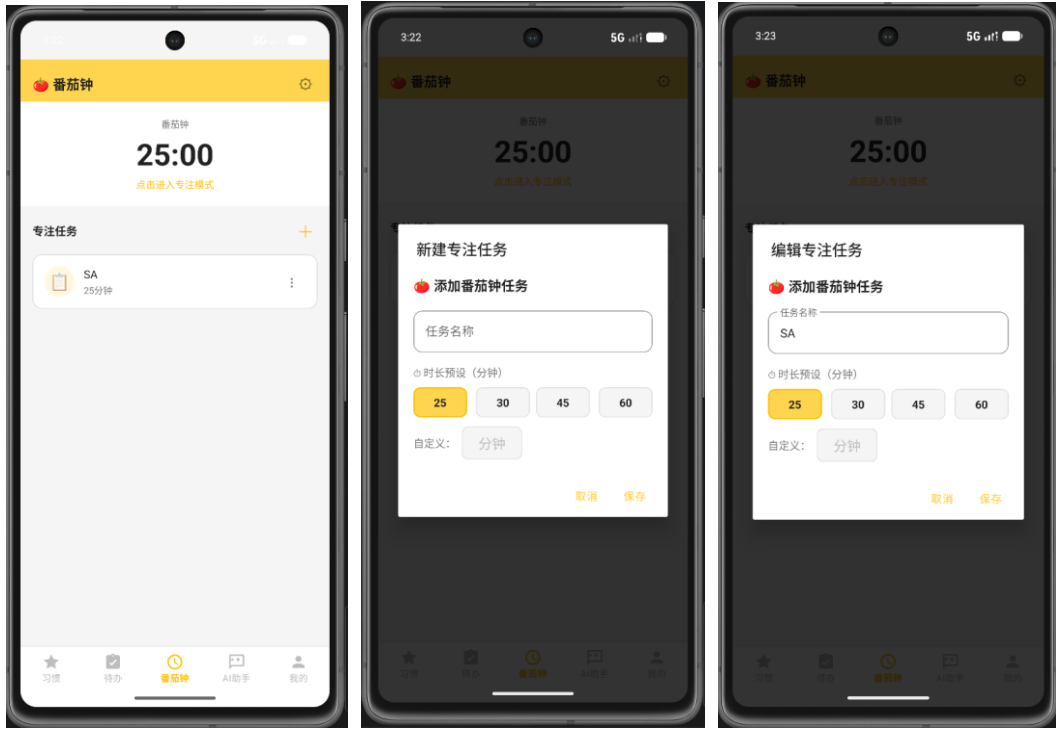


图 2-3-5 至图 2-3-7 默认番茄钟设置与新建、编辑专注任务

2.4 AI 智能助手

气泡式聊天界面，集成 DeepSeek Chat API。用户可用自然语言创建/修改/删除/查询待办、习惯和番茄钟任务（12 种命令）。AI 回复中的命令由本地 CommandParser 解析并直接操作 Room 数据库。支持多轮对话、消息持久化、API Key 热切换、清空对话、长按删除单条消息。

问题难点与实现路径：

DeepSeek API 集成： ChatClient 单例（双检锁）管理 Retrofit 客户端，60 秒超时，支持 resetInstance() 热切换 API Key。

CMD 协议：系统提示(System Prompt)注入命令格式规范，AI 回复中嵌入 [CMD]{"action":"...", "参数":"值"}[/CMD] 块，由 CommandParser.process() 正则提取并执行。

命令执行器： CommandParser 通过 DAO 层操作 Room 数据库，支持 12 种操作（create\_todo/delete\_todo/complete\_todo/update\_todo/list\_todos 等）。

中文星期解析： parseChineseDays() 支持"周一、二、四"和"124"两种格式。

重试机制：若 AI 回复不含命令但用户意图为操作请求，自动重试一次并附加强提醒。

```
public Result process(String aiText) {
    StringBuilder cleanedText = new StringBuilder(aiText);
    StringBuilder execLog = new StringBuilder();

    Matcher m = CMD_PATTERN.matcher(aiText);
    while (m.find()) {
        String json = m.group(1).trim();
        String outcome = execute(json);
        if (outcome != null) {
            execLog.append(outcome).append("\n");
        }
        // Remove the command block from displayed text
        int start = cleanedText.indexOf(m.group());
        int end = start + m.group().length();
        if (start >= 0) {
            cleanedText.replace(start, end, " ");
        }
    }

    String display = cleanedText.toString().trim();
    if (display.isEmpty()) {
        display = execLog.toString().trim();
    }

    return new Result(display, execLog.toString().trim());
}
```

图 2-4-1 CMD 协议解析代码  
第11页 共20页



图 2-4-2、图 2-4-3 AI 助手创建习惯



图 2-4-4、图 2-4-5 AI 助手修改待办

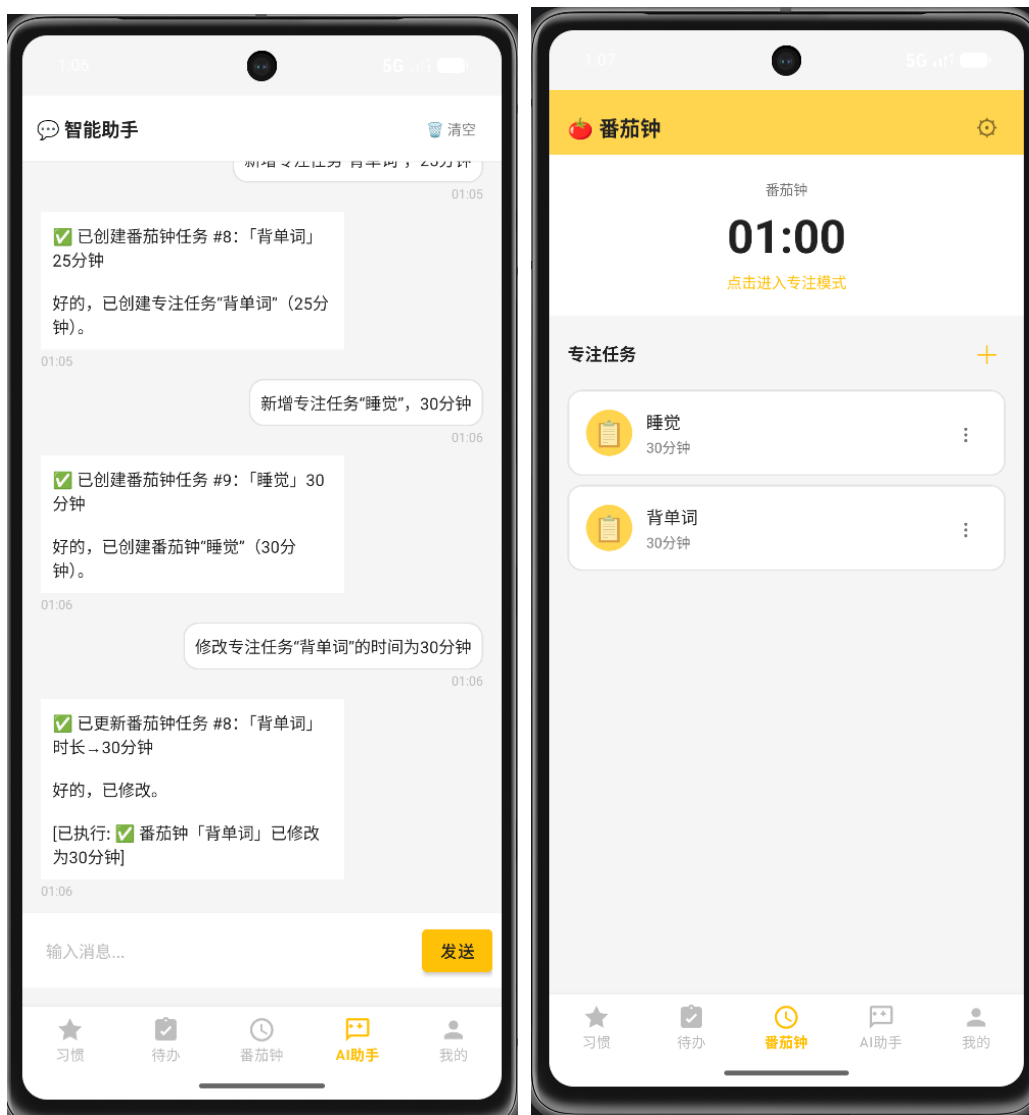


图 2-4-6、图 2-4-7 AI 助手创建、修改番茄钟

## 2.5 我的

个人中心包含用户登录/注册入口、40 款 emoji 头像选择、数据统计面板（习惯数/待办数/番茄完成次数）、位置卡片（后台自动定位 + LiveData 驱动刷新）、设置列表（AI 配置、数据备份管理、应用权限入口）。

### 关键技术点：

**密码安全：**登录密码 SHA-256 哈希存储（ HashUtils.java ）；"记住密码"功能使用 AndroidKeystore 硬件级 AES-256/GCM 加密（ CryptoUtils.java ），密钥不离开设备，卸载后自动销毁。

**数据备份：** BackupManager.java 将所有待办和习惯导出为 JSON，双存储策略（应用私有目录 files/backups/ + 公共 Downloads 文件夹）

**后台自动定位：**MainActivity 启动时自动调用 requestSingleLocation() ，15 秒超时自动降级到上次已知位置， LocationHelper 通过 LiveData 广播结果， MineFragment 观察并即时显示。



图 2-5-1 我的

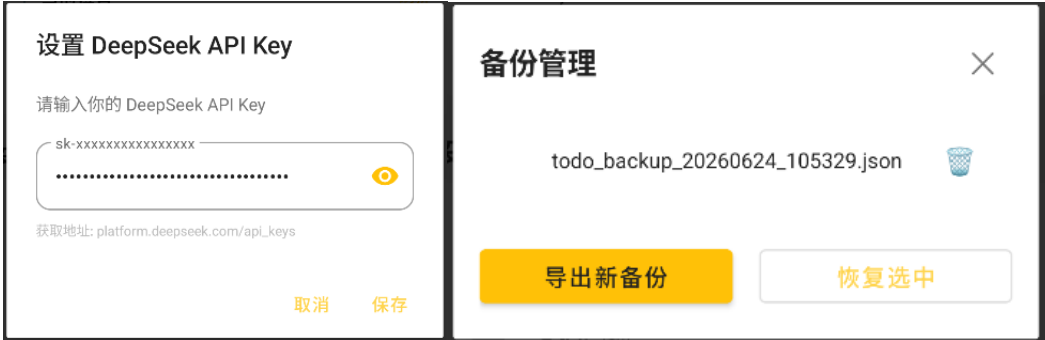


图 2-5-2、图 2-5-3 智能助手 APIKey 绑定与备份管理

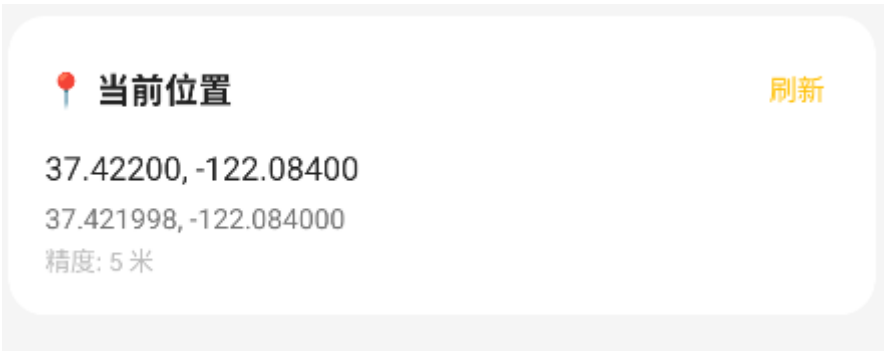


图 2-5-4 定位功能



图 2-5-5 习惯、待办数及专注任务完成数

## 2.6 其他特色功能

### 2.6.1 特色封面及语录

应用冷启动展示 2.5 秒过渡动画（600ms 淡入效果）。优先从 [api.quotable.io](https://api.quotable.io) 获取英文励志语录（OkHttp 同步请求，5 秒超时），网络失败则从本地预置的 15 条中文名言中随机选取。



图 2-6-1-1、图 2-6-1-2 封面及励志语录展示

### 2.6.2 本地智能推荐引擎

对于每个未在今天打卡的习惯，基于五个维度进行综合打分

- ①时间槽匹配（通过关键词字典将习惯映射到早/午/晚时段，匹配当前时段 +2.0 分，无关键词则 0 分中性，错配 -1.0 分）；
- ②近 7 天完成率（ $\geq 70\%$  +3.0 分，30%-70% 按比例折算， $< 30\%$  -1.0 分，无历史记录 0 分）；
- ③连续打卡奖励（ $\geq 5$  天 +2.0 分， $\geq 3$  天 +1.0 分）；
- ④荒废惩罚（距上次打卡  $\geq 7$  天 -2.0 分， $\geq 3$  天 -1.0 分，无历史记录不扣分）；
- ⑤今日已完成直接跳过不参与评分。

最终只有总分  $\geq 1.0$  的习惯入围，且最多推荐 2 个。

```
double score = 0.0;

// 2) Time-slot match
Integer preferredSlot = resolveTimeSlot(habit);
if (preferredSlot != null && preferredSlot == currentSlot) {
    score += 2.0; // matches current time slot
} else if (preferredSlot != null) {
    score -= 1.0; // mismatched slot -> mild penalty
}
// preferredSlot == null -> neutral (0), habit has no time association

// 3) 7-day completion rate
double rate = getRecentCompletionRate(habit.id, todayStamp, 7, userId);
if (rate >= 0.01) { // has at least some history
    if (rate >= 0.7) {
        score += 3.0; // strong habit
    } else if (rate >= 0.3) {
        score += rate * 3.0; // moderate habit (scaled)
    } else {
        score -= 1.0; // neglected habit (rate < 0.3)
    }
}
// No history -> neutral (0), don't give free points to new habits

// 4) Streak bonus: consecutive days completed (counting back from yesterday)
int streak = getConsecutiveStreak(habit.id, todayStamp, userId);
if (streak >= 5) {
    score += 2.0;
} else if (streak >= 3) {
    score += 1.0;
}

// 5) Neglect penalty: days since last completed check
int daysSinceLast = getDaysSinceLastCheck(habit.id, todayStamp, userId);
if (daysSinceLast >= 7) {
    score -= 2.0;
} else if (daysSinceLast >= 3) {
    score -= 1.0;
}

if (score >= 1.0) {
    scored.add(new ScoredHabit(habit, score));
}
```

图 2-6-2-1 推荐逻辑

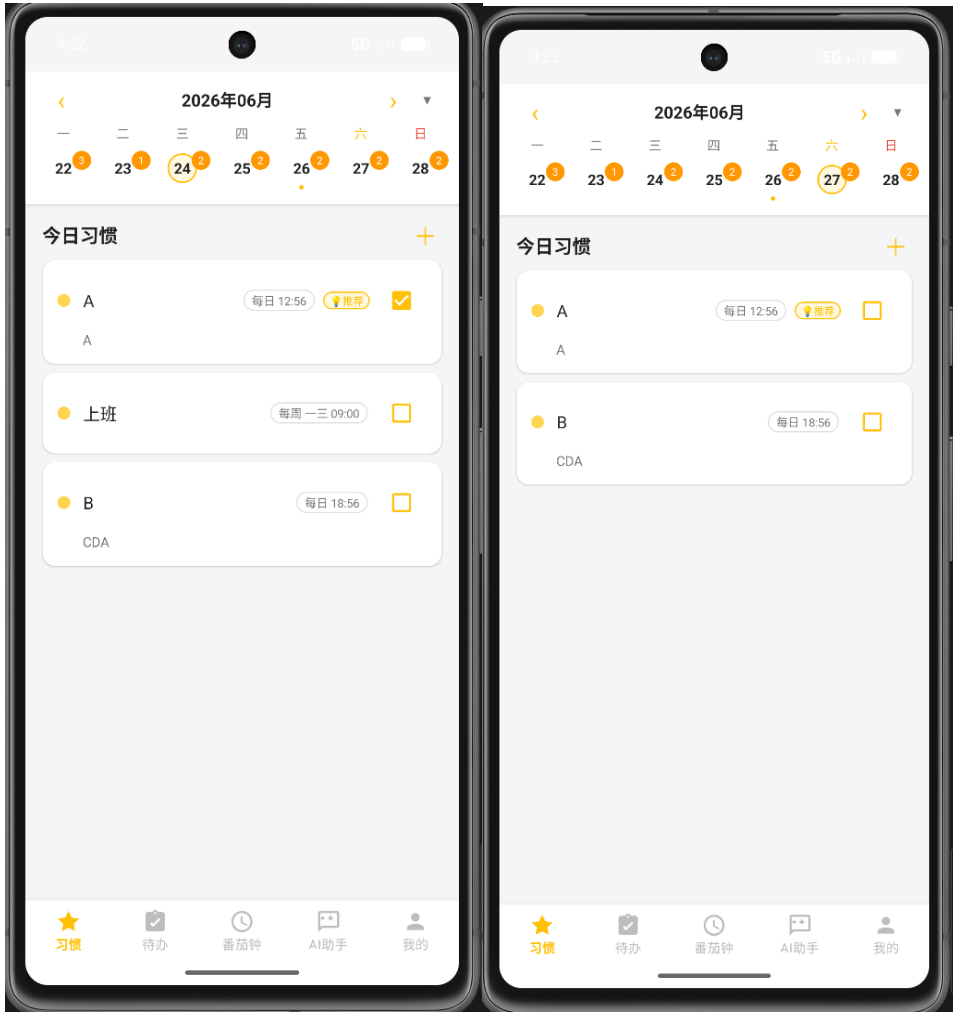


图 2-6-2-2、图 2-6-2-3 历史完成与推荐效果





图 2-7-1、图 2-7-2 注册、登录页与记住密码

## 四、实验总结

（此处写你的过程，比如遇到的错误，以及解决方法，你的所想、所得）

### 1.开发时遇到的问题与解决方案

#### ①AI 多轮对话中命令丢失

向 AI 助手发送"创建待办"等操作指令后，AI 有时只回复自然语言描述而不包含[CMD]命令块，导致操作无法被 CommandParser 解析执行。

原因分析：DeepSeek 大模型在多轮对话中可能"忘记"系统提示中的命令格式要求，尤其在对话历史增长后，模型倾向于只回复自然语言。

#### 解决方案：

1. 强化系统提示：将 CMD 协议说明放在系统提示的核心规则位置，设置最高优先级，并增加具体示例。
2. 自动重试机制：ChatFragment.java 中实现 userLikelyWantedAction() 方法，检测用户消息中是否包含"创建/删除/完成/修改/添加/番茄/习惯/待办"等关键词。若 AI 回复不含命令但用户意图明确为

操作请求，自动在对话上下文中追加强制提醒消息（“你刚才的回复没有包含 [CMD] 命令块！用户要求执行操作，你必须严格按格式发出命令。现在请仅回复命令（不要其他文字）：”，并调用 `retryCommand()` 重新请求 API。

3. 上下文保持：将完整 AI 回复（含 CMD 标签）存回 `conversation` 列表，避免模型因无法看到自己之前发过的命令格式而退化。

经验总结：提示工程（Prompt Engineering）在实际应用中的鲁棒性至关重要。不能完全依赖单次 API 调用返回正确格式，必须在本地实现兜底+重试机制。

```
return
// — P0: CRITICAL - operation commands —
"● 核心规则：当用户要求创建/删除/完成/查询待办、习惯或番茄钟时，你必须在回复末尾附加 [CMD]命令[/CMD]：\n" +
"没有命令的回复会导致操作无法执行。\\n\\n" +
"命令格式：[CMD]{\"action\": \"动作\", \"参数\": \"值\"}/[CMD]\n" +
"命令和自然语言写在同一消息里，命令放末尾。\\n\\n" +
"### 命令列表：\n" +
"  create_todo → title(可选), note(可选), priority(1低/2中/3高), due(可选, 格式yyyy-MM-dd HH:mm)\n" +
"  delete_todo → title(标题关键词) 或 id(编号)\n" +
"  complete_todo → title 或 id\n" +
"  list_todos → 无参数\n" +
"  create_habit → name, freq(每日/每周), days(每周必填, 数字1-7对应周一到周日, 如\"124\"=周一/二/四), time(可选, HH:mm如09:00), desc(可选)\n" +
"  delete_habit → name 或 id\n" +
"  list_habits → 无参数\n" +
"  create_pomo → name, minutes(如25)\n" +
"  delete_pomo → name 或 id\n" +
"  update_todo → 必须指定 title(要修改的待办标题关键词, 用于匹配) 或 id, 可选: new_title(新标题), note, priority(1-3), due(yyyy-MM-dd)\n" +
"  update_habit → 必须指定 name(要修改的习惯名称关键词, 用于匹配) 或 id, 可选: new_name(新名称), desc, icon, color, freq(每日/每周), time(HH:mm), days(如\"135\")\n" +
"  update_pomo → 必须指定 name(要修改的番茄钟名称关键词, 用于匹配) 或 id, 可选: new_name(新名称), icon, minutes\n" +
"  list_pomos → 无参数\n" +
"### 示例：\n" +
"用户：「创建待办买菜」→ 你：「好的，已创建。[CMD]{\"action\": \"create_todo\", \"title\": \"买菜\", \"priority\": 1}/[CMD]」\n" +
"用户：「删除买菜」→ 你：「好的。[CMD]{\"action\": \"delete_todo\", \"title\": \"买菜\"}/[CMD]」\n" +
"用户：「每天7点晨跑」→ 你：「好的。[CMD]{\"action\": \"create_habit\", \"name\": \"晨跑\", \"freq\": \"每日\", \"time\": \"07:00\"}/[CMD]」\n" +
"用户：「每周二四六早上8点晨跑」→ 你：「好的。[CMD]{\"action\": \"create_habit\", \"name\": \"晨跑\", \"freq\": \"每周\", \"days\": \"135\", \"time\": \"08:00\"}/[CMD]」\n" +
"用户：「把买菜改成买菜和水果」→ 你：「好的。[CMD]{\"action\": \"update_todo\", \"title\": \"买菜\", \"new_title\": \"买菜和水果\"}/[CMD]」\n" +
"用户：「修改晨跑习惯为每天9点」→ 你：「好的。[CMD]{\"action\": \"update_habit\", \"name\": \"晨跑\", \"freq\": \"每日\", \"time\": \"09:00\"}/[CMD]」\n" +
"用户：「把学习任务改成45分钟」→ 你：「好的。[CMD]{\"action\": \"update_pomo\", \"name\": \"学习\", \"minutes\": 45}/[CMD]」\n" +
"📅 今天= " + todayStr + " " + weekday +
" 明天= " + (tomorrow.getMonthValue() + "月" + tomorrow.getDayOfMonth() + "日") +
" 后天= " + (today.plusDays(2).getMonthValue() + "月" + today.plusDays(2).getDayOfMonth() + "日") + "\n" +
"回复简短(≤100字)，命令放在末尾。\\n\\n" +
// — P1: Software overview —
"### 关于本软件\n" +
"你是「待办清单」App内置助手。5个标签：习惯、待办、番茄钟、AI助手、我的。\\n" +
"· 习惯：日历+打卡，每日/每周，可设提醒\n" +
"· 待办：优先级+截止日期，左滑删除，勾选完成\n" +
"· 番茄钟：倒计时，前台服务，通知栏操作\n" +
"· 我的：头像/统计/备份/密码锁/API设置\n" +
"所有数据本地存储(Room)。";
}
```

图 4-1、图 4-2 CMD 协议说明 prompt

## ②每周习惯的星期参数解析

命令 AI 创建“每周一、三、五”习惯时，系统只存储“周一三五”的文本描述，无法正确解析为数字格式“135”。

### 解决方案：

在 `CommandParser.parseDays()` 方法中实现双格式兼容：

1. 数字格式：直接过滤非 1-7 字符，保留纯数字串（如“135”）。
2. 中文格式：`parseChineseDays()` 方法遍历 21 种中文星期表达（“周一”/“星期一”/“周 1”等），检测文本中的匹配模式，映射为对应数字。
3. 双重后备：若 `days` 参数为空，回退到扫描 `desc` 描述字段中的中文星期名称。

```
private static String parseChineseDays(String text) {
    StringBuilder result = new StringBuilder();
    // Map: Chinese day names to digit
    String[] patterns = {"周一", "周二", "周三", "周四", "周五", "周六", "周日",
        "星期一", "星期二", "星期三", "星期四", "星期五", "星期六", "星期日",
        "周1", "周2", "周3", "周4", "周5", "周6", "周7"};
    int[] digits = {1, 2, 3, 4, 5, 6, 7,
        1, 2, 3, 4, 5, 6, 7,
        1, 2, 3, 4, 5, 6, 7};
    for (int i = 0; i < patterns.length; i++) {
        if (text.contains(patterns[i]) && result.indexOf(String.valueOf(digits[i])) < 0) {
            result.append(digits[i]);
        }
    }
    return result.toString();
}
```

图 4-3 日期解析代码

### ③定位请求长时间无响应

在某些设备或室内环境下，GPS 定位请求长时间无返回，导致“我的”页面一直显示“正在定位...”。

**解决方案：**

在 `LocationHelper.requestSingleLocation()` 中实现 15 秒超时机制：

```
final Runnable timeoutRunnable = () -> {
    Log.w(TAG, msg: "requestSingleLocation timed out, falling back to last location");
    stopTracking();
    getLastLocation(context);
    LocationLiveData.postValue(LocationInfo.status(msg: "定位超时，显示上次缓存位置"));
};
timeoutHandler.postDelayed(timeoutRunnable, delayMillis: 15_000);
```

图 4-4 定位超时机制

同时在 `MainActivity.onCreate()` 中启动定位，利用应用冷启动到用户切换到“我的”页面的时间窗口，使得切换到“我的”时定位通常已完成。

## 2.实验过程回顾

本项目从零开始构建了一款完整的 Android 时间管理应用，涵盖了移动互联网应用开发的大量技术栈。开发过程经历了以下阶段：

- ①需求分析与架构设计：撰写 PRD 文档，确定功能模块、技术架构设计、数据库表结构
- ②基础框架搭建：配置 Gradle 构建系统、Room 数据库、导航结构、主题样式
- ③核心功能开发：依次实现待办 CRUD、习惯打卡、番茄钟前台服务、AI 聊天助手、个人中心
- ④特色功能完善：CMD 命令协议、推荐引擎、定位服务、备份管理
- ⑤调试与问题修复：解决 AI 命令丢失、日历角标错乱、番茄钟状态恢复、定位超时等问题
- ⑥测试与优化：性能优化、代码审查

## 3.不足之处与可改进方向

- ①AI 对话体验：当前命令协议需完全依赖 AI 生成正确 JSON，对复杂意图（如“把昨天的未完成待办全部移到今天”）支持不足。后续可引入 Function Calling 机制，由本地定义函数 schema 让模型直接调用，提高可靠性。
- ②数据库迁移：当前使用 `fallbackToDestructiveMigration()` 简单粗暴，升级时数据丢失。后续应实现 Room Migration 对象进行平滑升级。
- ③同步与云端能力：当前所有数据仅存储在本地，无云端同步功能。后续可接入 Firebase Firestore 或自建后端实现多设备数据同步。
- ④测试覆盖：当前仅有 2 个占位测试文件，缺乏单元测试和 UI 测试。后续应对 `CommandParser`、`RecommendationEngine`、`BackupManager` 等核心逻辑类补充 JUnit 测试。

#### 4.实验心得与体会

通过本次《移动互联网应用》课程设计项目的完整开发实践，我获得了以下几点重要收获：

（1）Android 组件化开发的系统性理解。本项目涉及了 Activity、Fragment、Service、BroadcastReceiver、ContentProvider 四大组件以及 Room、SharedPreferences、Android Keystore 等多种持久化方式。通过实际编码，我深刻理解了各组件的生命周期、通信方式(Intent/Broadcast/LiveData)以及适用场景。

（2）前后台协同与进程保活。番茄钟功能要求应用在后台甚至进程被杀死后仍能正确计时，这促使我深入研究了 Android 前台服务机制、START\_STICKY 策略以及 SharedPreferences 状态持久化的组合方案。

（3）提示工程与大模型集成实践。AI 助手功能的开发让我认识到，将大模型集成到实际产品中不仅仅是调用 API，还需要设计鲁棒的协议（CMD）、考虑失败重试、处理格式异常等工程问题。在实际应用中，永远不能完全信任 AI 响应的正确性，必须在本地实现校验和兜底机制。

（4）防御性编程的重要性。Android 生态碎片化严重，不同设备、不同 Android 版本的 API 行为可能完全不同。本项目中的定位降级、Geocoder 兼容处理、LocalDate.now() 异常保护等细节，都是防御性编程的具体体现。

（5）软件工程的全流程视角。从 PRD 撰写、架构设计、编码实现到问题排查，完成了一次完整的软件开发流程，加深了对"需求→设计→实现→测试"这一软件工程核心循环的理解。

（6）安全意识的建立。在实现登录和数据保护功能时，学习了 SHA-256 哈希、AES-256/GCM 对称加密、Android Keystore 硬件安全模块等实用安全技术，意识到移动端数据安全不仅仅是"加密"二字，更需要合理的密钥管理和安全存储方案。